

My Technical Stack & Practical Experience

Kalyani M G

1. Development Environment & Coding Tools

- **Visual Studio Code**

What it is: A lightweight and powerful code editor used for software development.

How I use it: I use VS Code for Python projects, web development, debugging, and managing project folders. I regularly use extensions like Python, Live Server, and GitHub tools.

My familiarity: High

Challenges faced: Initially understanding debugging tools and configuring extensions properly.

Benefits: Fast, customizable, and suitable for multiple programming languages.

- **Google Colab**

What it is: A cloud-based Jupyter notebook platform with free GPU support.

How I use it: I use Colab for machine learning model training, deep learning experiments, and running heavy Python notebooks.

My familiarity: High

Challenges faced: Session timeout issues and managing runtime memory.

Benefits: No local setup required, GPU access, easy notebook sharing.

- **GitHub**

What it is: Platform for version control and hosting code repositories.

How I use it: I upload projects, maintain code versions, create README files, and share projects publicly.

My familiarity: Medium to High

Challenges faced: Understanding Git commands and branch workflows in the beginning.

Benefits: Essential for collaboration and project portfolio building.

2. Web Frameworks & Frontend

- **Streamlit**

What it is: Python framework for building interactive web apps quickly.

How I use it: I used Streamlit to create chatbot interfaces and AI dashboards.

My familiarity: Low

Challenges faced: Custom UI styling limitations.

Benefits: Very fast deployment for AI projects.

- **Flask**

What it is: Lightweight Python backend web framework.

How I use it: Used Flask for learning APIs, routing, and backend logic integration.

My familiarity: Medium

Challenges faced: Understanding project structure and templates.

Benefits: Good for beginners and backend fundamentals.

- **HTML / CSS / JavaScript**

How I use them: Used for building static websites, landing pages, and UI design.
My familiarity: Medium
Challenges faced: Responsive design and alignment issues.
Benefits: Core technologies for web development.

3. AI / Machine Learning Frameworks 🤖

- **Scikit-learn**

What it is: Python library for machine learning models.
How I use it: Used for classification, regression, preprocessing, and model evaluation.
My familiarity: Low
Challenges faced: Selecting correct models and tuning parameters.
Benefits: Easy and efficient for ML projects.

- **TensorFlow**

How I use it: Used for neural network learning and deep learning basics.
My familiarity: Medium
Challenges faced: Complex syntax compared to simpler ML libraries.
Benefits: Strong industry-level deep learning framework.

- **PyTorch**

How I use it: Used for neural network experiments and academic tasks.
My familiarity: Medium
Challenges faced: Tensor operations and architecture setup.
Benefits: Flexible and research-friendly.

- **Pandas / NumPy**

How I use them: Data cleaning, transformation, numerical operations.
My familiarity: High
Benefits: Essential tools for data science workflows.

4. Agentic AI Tools ⚡

- **LangChain**

What it is: Framework for building LLM-powered workflows and agents.
How I use it: Used in chatbot projects connecting LLMs with prompts and memory.
My familiarity: Low
Challenges faced: Understanding chains and agent logic.
Benefits: Powerful for real AI assistants.

- **CrewAI**

How I use it: Explored multi-agent workflows where different AI agents perform tasks collaboratively.
My familiarity: Low
Benefits: Good introduction to autonomous AI systems.

5. LLM Platforms 🧠

- **OpenAI**

What it is: A leading AI platform that provides advanced large language models such as GPT models for text generation, coding, reasoning, and multimodal tasks.

How I use it: I use OpenAI tools mainly through ChatGPT for coding help, debugging, technical learning, documentation writing, and project idea generation. I have also explored API-based usage concepts for chatbot development.

My familiarity: High

Challenges faced: Learning prompt engineering techniques to get accurate and efficient responses, and understanding API usage concepts.

Benefits: Strong reasoning ability, coding assistance, productivity support, and broad use cases for students and developers.

- **Hugging Face**

What it is: A popular AI ecosystem that hosts open-source models, datasets, and transformer libraries.

How I use it: I use Hugging Face to explore pre-trained NLP models, experiment with transformers, and understand modern open-source AI workflows.

My familiarity: Medium

Challenges faced: Choosing suitable models from a very large library and understanding model resource requirements.

Benefits: Huge model ecosystem, strong community support, and valuable for experimentation and learning.

6. AI Productivity Tools 🛠️

- **ChatGPT**

What it is: A conversational AI assistant used for learning, coding, writing, and problem-solving.

How I use it: I regularly use ChatGPT for debugging code, understanding technical concepts, improving project documentation, and brainstorming ideas for academic or personal projects.

My familiarity: High

Challenges faced: Learning how to write clear prompts to get accurate and useful responses.

Benefits: Saves time, improves productivity, and acts as a learning companion for multiple subjects.

- **Grammarly**

What it is: An AI writing assistant for grammar, spelling, and clarity improvement.

How I use it: I use it to improve emails, reports, LinkedIn content, and formal writing.

My familiarity: High

Challenges faced: Sometimes suggestions need manual review for tone.

Benefits: Improves communication quality and professionalism.

7. Cloud Platforms ☁️

- **Google Colab / Google Cloud Ecosystem**

What it is: Cloud-based computing environment mainly used for notebooks and machine learning workloads.

How I use it: I use Google Colab for running Python notebooks, training models, testing datasets, and using GPU resources.

My familiarity: High

Challenges faced: Runtime disconnections and storage/session limits.

Benefits: Easy access to compute power without high-end local hardware.

- **Amazon Web Services (AWS)**

What it is: A leading cloud platform offering storage, compute, and deployment services.

How I use it: I am learning AWS fundamentals such as EC2, S3, and cloud deployment concepts.

My familiarity: Beginner to Medium

Challenges faced: Large number of services can feel overwhelming initially.

Benefits: Strong industry relevance and scalable cloud infrastructure.

- **Microsoft Azure**

What it is: Enterprise cloud platform by Microsoft.

How I use it: Basic exposure through learning cloud concepts and AI service exploration.

My familiarity: Beginner

Challenges faced: Understanding service mapping compared to AWS/GCP.

Benefits: Useful for enterprise systems and Microsoft-based environments.

- **Streamlit Community Cloud**

What it is: Hosting platform for Streamlit apps.

How I use it: Used for deploying small AI web apps and demos online.

My familiarity: Medium

Challenges faced: Resource limits for larger applications.

Benefits: Quick and beginner-friendly deployment.

8. Databases

- **MySQL**

What it is: A relational database management system using SQL.

How I use it: I use MySQL for DBMS lab work, table creation, joins, normalization practice, and structured data storage.

My familiarity: High

Challenges faced: Writing complex joins and nested queries initially.

Benefits: Strong foundation for backend and enterprise database systems.

- **MongoDB**

What it is: A NoSQL document-oriented database.

How I use it: I used MongoDB while learning flexible schema storage and JSON-style documents.

My familiarity: Medium

Challenges faced: Understanding when NoSQL is better than relational databases.

Benefits: Good for unstructured and rapidly changing data.

- **Firebase**

What it is: Backend platform providing real-time databases and authentication tools.

How I use it: Explored Firebase for beginner web/mobile backend projects.

My familiarity: Beginner to Medium

Challenges faced: Managing structure efficiently in NoSQL databases.

Benefits: Fast backend setup with authentication and hosting features.